Kognify AI: Agentic Research Assistant

Experiential Learning Project Report: Language Processing Applications using ML, NLP, Agentic AI and LLMs

SSS1. Title and Group Details

Project Title: Kognify AI – An Agentic Research Assistant for Academic Literature Discovery, Verification and Citation

Group Number: 10

GitHub Repository: https://www.github.com/hyon99/Kognify.AI

Deployed Application URL: https://kognify-ai.streamlit.app

S.No.	Name	Roll Number	Role	Contribution
1	Abhrajyoti Das	2304065	Front End Development	Designed and built the Streamlit user interface (`app.py`): chat workspace, Library, Plugins and Explore pages, theming and responsive layout, session handling in the UI.
2	Sudipon Makal	2304127	Agent Development and Website Deployment	Built the Google ADK agent (`agent.py`): OpenAlex tools, system prompt, rate limiting, APA 7 bibliography generator; integrated the agent with the UI through the ADK Runner; deployed the web application.
3	Amlan Kumar Sinha	2304140	Documentation	Prepared this report, block diagrams, README, setup instructions, sample inputs/outputs and the presentation video script.
4	Archisman Das	2304079	Testing	Prepared test cases, tested each tool and the end-to-end workflow, error and rate-limit handling, UI behaviour, and recorded observations and limitations.
5	Soumyadipta Bose	2304158	Video & Presentation	Prepared Presentation and Video Creation for the final submission of the work.

2. Problem Statement and Objectives

2.1 Description of the Problem

Researchers, students and engineers spend a large share of their time on literature review: finding relevant papers, reading abstracts, checking whether a paper really exists, comparing approaches and formatting references. General-purpose chatbots can summarise topics quickly, but they are known to **hallucinate** papers, authors, DOIs and results. A reliable research assistant must instead ground every claim in real, retrievable scholarly records and be transparent about what it could not find.

The selected problem is **Agentic Research Assistant**: build a prototype in which an LLM-driven agent plans a research task, calls external scholarly tools, and returns a structured, verifiable literature summary with references.

2.2 Motivation

- Manual literature search across many sources is slow and repetitive.
- LLMs alone fabricate citations; grounding them in a scholarly index reduces this risk.
- Agentic AI (plan, act with tools, observe, respond) suits a multi-step workflow such as search, select, retrieve, verify, synthesise and cite.
- Free, open scholarly metadata (OpenAlex) makes a low-cost, open-source-friendly prototype possible.

2.3 Objectives

1. Build an LLM agent that autonomously plans and executes a multi-step literature search.
2. Retrieve real paper metadata and abstracts from the OpenAlex API rather than from the model's memory.
3. Confirm that each cited paper exists as an OpenAlex record and report what that does and does not prove.
4. Generate correctly formatted APA 7 bibliographies without relying on the LLM to write citations.
5. Provide a clean, usable chat-based web interface with a personal library of saved answers.
6. Handle API limits and missing data transparently instead of silently inventing content.

2.4 Expected Outcomes

- A working web application where a user types a research question and receives a structured literature review with sources.
- Responses that clearly mark unavailable abstracts or metadata.
- Ready-to-use APA 7 reference lists.
- A documented, reproducible GitHub project.

3. Proposed Methodology and Processing Pipeline

3.1 System Overview

Kognify AI has two layers. The **presentation layer** is a Streamlit web application (`app.py`). The **agent layer** is a Google Agent Development Kit (ADK) agent (`research_assistant/agent.py`) powered by a

Gemini model, which is given four Python tools that call the OpenAlex REST API. The UI calls the agent through an ADK `Runner` with an in-memory session service, so each chat keeps conversational context.

3.2 Block Diagram of the Processing Pipeline

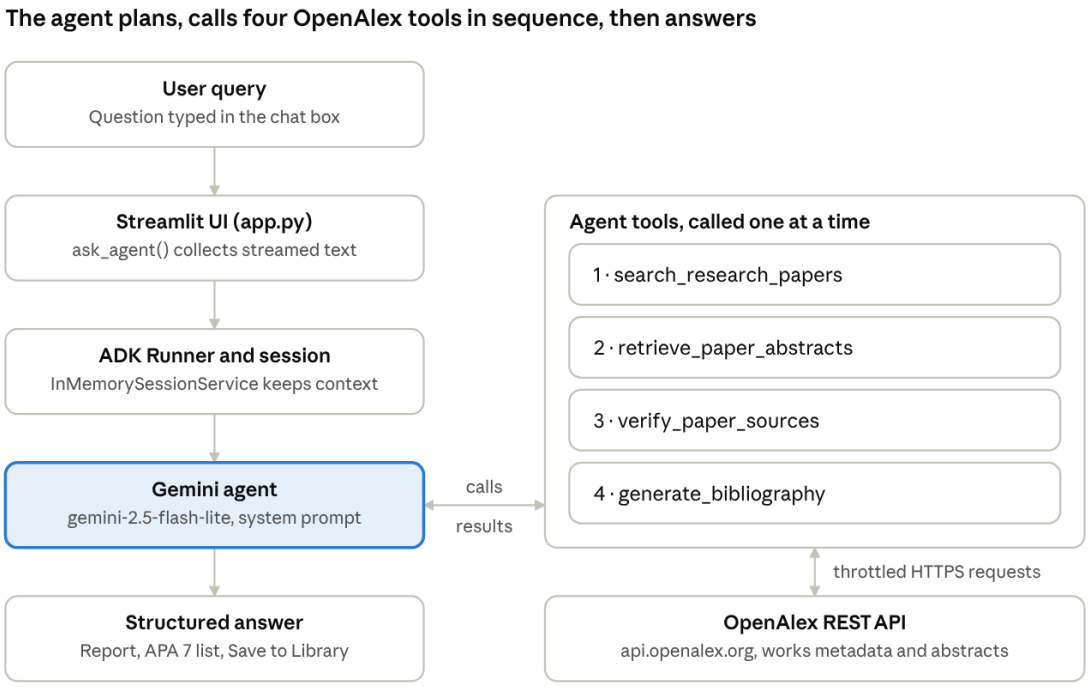


Figure 1. A question flows down the left column into the Gemini agent, which calls the four OpenAlex tools on the right and returns a structured answer.

3.3 Step-by-Step Explanation of the Processing Stages

The agent's behaviour is governed by a system prompt that defines a seven-stage workflow.

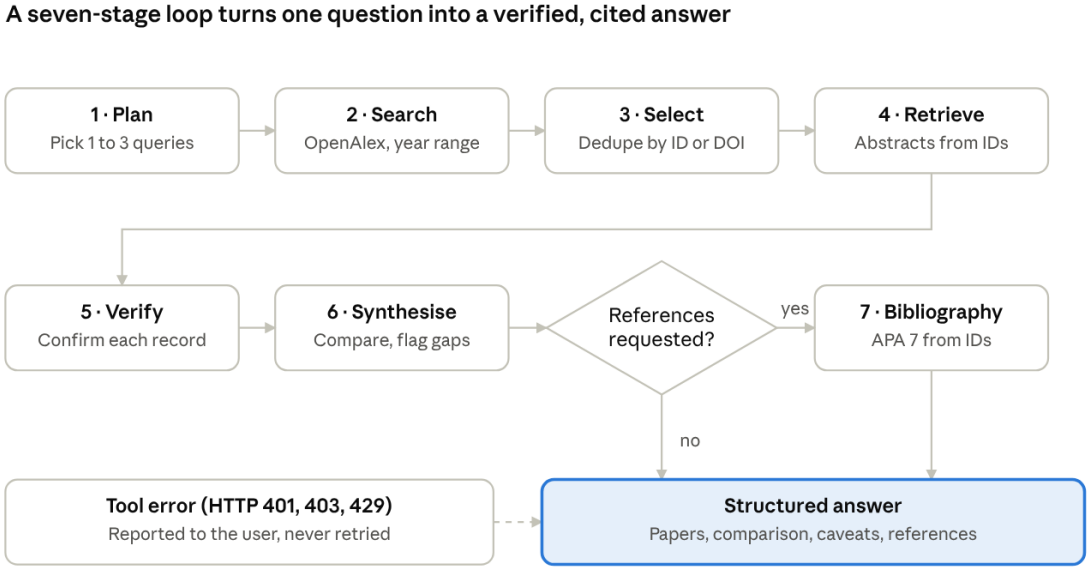


Figure 2. The agent works through stages 1 to 6 in order, adds a bibliography only when references are requested, and reports tool errors instead of retrying them.

Stage	Name	What happens	Tool / component
1	Plan	The agent states a short plan and chooses 1 to 3 focused search queries from the user's question.	Gemini LLM
2	Search	Queries are run one after another against OpenAlex with a publication-year filter (default 2020 to 2026).	`search_research_papers`
3	Select	Results are deduplicated by OpenAlex ID or DOI; relevant papers are shortlisted.	Gemini LLM
4	Retrieve	Full metadata is fetched for the shortlisted IDs and the abstract is rebuilt from OpenAlex's inverted index.	`retrieve_paper_abstracts`
5	Verify	Each ID is re-checked against OpenAlex; DOI and venue availability is reported. The result is explicitly marked as *not* independent publisher or peer-review verification.	`verify_paper_sources`
6	Synthesise	The agent compares methods, findings and limitations, only where supported by retrieved abstracts or metadata, and flags missing abstracts.	Gemini LLM

Stage	Name	What happens	Tool / component
7	Bibliography	When references are requested, the agent passes only a list of OpenAlex IDs; the tool fetches the records itself and formats them in APA 7.	`generate_bibliography`

3.4 Input, Intermediate Processing and Output

Input: A natural-language research request typed in the chat box, for example *"Find recent research on lightweight object detection for edge devices."* Optional preferences such as year range, number of papers, or a request for references can be included in the text.

Intermediate processing:

- The LLM converts the request into search keywords and a year range.
- Each tool call is throttled (model calls and OpenAlex calls both have minimum intervals) and returns a structured JSON dictionary with a `status` field.
- The abstract is reconstructed from OpenAlex's `abstract_inverted_index` by sorting words by their position offsets.
- Author names are converted to APA form (surname plus initials), with the APA 7 rule of listing the first 19 authors, an ellipsis and the last author when there are more than 20.

Output: A structured markdown response containing the plan, the selected papers (title, authors, year, venue, DOI, citation count), abstract-based comparison, verification notes, stated limitations and, on request, an APA 7 bibliography. Users can save any response to the in-app Library.

3.5 Design Decisions that Reduce Hallucination

- The prompt forbids inventing papers, authors, abstracts, DOIs, venues or metrics.
- Bibliographies are produced by deterministic code, not by the LLM. The model never writes metadata as JSON, which removes formatting errors.
- Tool calls are sequential, and the agent must not retry 401, 403 or 429 errors repeatedly.
- Tools return explicit error or `no_results` statuses rather than empty lists, so the agent can report failures honestly.

4. Models and Tools Used

4.1 Language Model

Item	Details
Model	Google Gemini `gemini-2.5-flash-lite` (default; configurable through the `ADK_MODEL` environment variable)
Source	Google AI (Gemini API), accessed with a `GOOGLE_API_KEY`: https://ai.google.dev/gemini-api/docs

Item	Details
Role	Planning, query formulation, paper selection, tool orchestration and synthesis of abstracts
Hugging Face models	Not used. The system relies on a hosted LLM and a scholarly API; no model weights are downloaded.

4.2 Agentic and Retrieval Approach

The project follows an **agentic, tool-augmented retrieval-grounded generation** design. Instead of embedding documents in a vector database, the agent retrieves evidence live from OpenAlex at question time and conditions its answer on the returned records. This is the same principle as RAG (ground the generation in retrieved sources) but uses API retrieval rather than a local vector index.

4.3 Data Source

OpenAlex is an open catalogue of scholarly works, authors, venues and institutions: https://openalex.org (API: https://api.openalex.org). It provides titles, authors, years, venues, DOIs, citation counts, open-access links and inverted-index abstracts.

4.4 Libraries and Frameworks

Component	Library / Tool	Purpose
Agent framework	Google ADK (`google-adk`): `Agent`, `Runner`, `InMemorySessionService`	Defines the agent, tools, callbacks and runs conversations
LLM SDK	`google-genai` (`types.Content`, `types.Part`)	Message objects sent to the agent
Web UI	Streamlit	Chat interface, navigation, session state
HTTP	`requests` (with a shared `Session`)	Calls to the OpenAlex REST API
Configuration	`python-dotenv`	Loads API keys and settings from `.env`
Concurrency	`asyncio`, `threading`	Async agent streaming; locks for rate limiting
Text processing	`re`, `json`	ID validation, name and abstract processing
Styling	Custom CSS (DM Sans and Playfair Display fonts)	Gradient workspace and dark sidebar

4.5 Implementation Environment

Python 3.10 or later, any OS, internet access, a Google AI API key, and optionally an OpenAlex API key and e-mail for the polite pool.

4.6 Configuration Parameters

Environment variable	Default	Meaning
`GOOGLE_API_KEY`	required	Gemini API key
`ADK_MODEL`	gemini-2.5-flash-lite	LLM used by the agent
`OPENALEX_API_KEY` / `OPENALEX_EMAIL`	empty	Optional OpenAlex credentials
`MODEL_MIN_INTERVAL_SECONDS`	4	Minimum gap between LLM requests
`OPENALEX_MIN_INTERVAL_SECONDS`	3	Minimum gap between OpenAlex requests
`MAX_SEARCH_RESULTS`	6 (range 1 to 20)	Cap on papers per call
`REQUEST_TIMEOUT_SECONDS`	20 (range 5 to 60)	HTTP timeout
`ADK_AGENT_MODULE`	research_assistant.agent	Module the UI imports the agent from

5. Implementation and Usability

5.1 Implementation Procedure

Step 1: Agent tools (`agent.py`). Four Python functions are exposed to the LLM as tools. Their docstrings and type hints let ADK generate the function-calling schema automatically.

Tool	Function
`search_research_papers(query, max_results, start_year, end_year)`	Keyword search on OpenAlex `works` with a date filter; returns a list of normalised paper records
`retrieve_paper_abstracts(openalex_ids)`	Fetches each work by ID and reconstructs the abstract
`verify_paper_sources(openalex_ids)`	Confirms each record exists in OpenAlex and reports DOI, venue and URL; sets `independently_publisher_verified` to false
`generate_bibliography(openalex_ids, style)`	Fetches records and builds an alphabetically sorted APA 7 list; reports metadata warnings and retrieval errors

Step 2: Reliability layer. `rate_limit_before_model` is registered as a `before_model_callback` and enforces a minimum interval between LLM calls. `_openalex_get` uses a lock to space API requests, attaches optional credentials, and raises clear errors for HTTP 401, 403 and 429.

Step 3: Normalisation. `_record` converts raw OpenAlex JSON into a uniform dictionary (authors, year, venue, volume, issue, pages, DOI, open-access URL, citations, abstract and `abstract_status`).

Step 4: Agent definition. `root_agent = Agent(...)` combines the model, the system prompt, the four tools and the rate-limit callback.

Step 5: UI bridge (`app.py`). `get_runner()` imports `root_agent`, creates an ADK `Runner` with an `InMemorySessionService`, and caches it in Streamlit session state. `ask_agent()` creates a session once per chat, sends the user's message and collects text parts from the streamed events into one answer.

Step 6: User interface. A two-panel layout: a dark sidebar (New chat, Scheduled, Library, Plugins, Explore) and an animated gradient workspace with the chat. Each assistant answer has a *Save to Library* button. The Plugins page lists the four agent capabilities, and the Explore page offers one-click starter research prompts. If the agent cannot be loaded, the UI shows a readable error with troubleshooting hints instead of crashing.

Step 7: Deployment. The Streamlit application is deployed as a website (for example on Streamlit Community Cloud or another Python host) with the API keys stored as platform secrets rather than in the repository.

5.2 Instructions for Running the Application

```
# 1. Clone the repository
git clone <repository-url>
cd <repository-folder>

# 2. Create and activate a virtual environment
python -m venv venv
source venv/bin/activate        # Windows: venv\Scripts\activate

# 3. Install dependencies
pip install streamlit google-adk google-genai requests python-dotenv

# 4. Create a .env file next to app.py
GOOGLE_API_KEY=your_gemini_key
OPENALEX_EMAIL=you@example.com      # optional
OPENALEX_API_KEY=your_key           # optional

# 5. Folder layout
#    app.py
#    .env
#    research_assistant/__init__.py
#    research_assistant/agent.py

# 6. Run the web app
streamlit run app.py
```

```
# (Optional) test the agent alone in the ADK developer UI
adk web
```

5.3 Practical Applications and Usability

- **Students:** quickly find recent papers for a seminar, thesis or mini-project.
- **Researchers:** run an initial literature scan and spot research gaps.
- **Project teams:** collect and format references in APA 7.
- **Educators:** teach agentic AI, tool use and grounded generation with a small, readable codebase.

Usability features: a single chat box, starter prompts, a Library for saved answers, transparent notes on missing data, and plain-language error messages.

6. Results and Sample Outputs

> ***Note for the team:*** *Replace the bracketed placeholders below with real screenshots and transcripts captured from your own runs. The tables describe what the system is designed to return; fill in the Observed column from actual testing.*

6.1 Representative Inputs and Outputs

#	Representative input	Expected output structure
1	Find recent research on lightweight object detection for edge devices.	Short plan, 3 to 6 OpenAlex papers with authors, year, venue, DOI and citation counts, abstract-based comparison, limitations
2	Compare transformer-based and CNN-based image classification methods.	Papers for each family, comparison of methods and findings drawn from abstracts, missing-abstract flags
3	Identify research gaps in multi-agent systems for academic literature review.	Selected papers plus gaps stated only where supported by retrieved text
4	Find papers on retrieval-augmented generation and give me an APA reference list.	Papers followed by an alphabetically sorted APA 7 bibliography from `generate_bibliography`
5	Give me references in IEEE style.	Message that this implementation currently supports APA 7 only

Illustrative format of an APA 7 entry produced by the bibliography tool:

```
Surname, A. B., & Surname, C. (Year). Title of the paper. *Journal Name*, *Volume*(Issue), FirstPage–LastPage. https://doi.org/xx.xxxx/xxxxx
```

Screenshots to insert:

- [Insert screenshot: home screen with the hero text and chat box]
- [Insert screenshot: a complete research response for Input 1]

- [Insert screenshot: bibliography output for Input 4]
- [Insert screenshot: Library page with a saved response]
- [Insert screenshot: Plugins and Explore pages]

6.2 Test Cases and Performance Evaluation

ID	Test	Expected behaviour	Observed	Pass/Fail
T1	Normal topic query	Returns up to the configured number of papers with metadata		
T2	Empty query to search tool	Returns an error message: "Query cannot be empty."		
T3	`start_year` greater than `end_year`	Returns a clear validation error		
T4	Paper with no abstract	Marked `not_available_in_openalex`; not invented		
T5	Invalid OpenAlex ID	Reported as `invalid_id`; other IDs still processed		
T6	Verification	Status `openalex_record_confirmed` with the publisher-verification caveat		
T7	Unsupported citation style	`unsupported_style` response; APA 7 offered		
T8	Missing metadata in a record	Bibliography entry produced with a `metadata_warnings` note		
T9	Rate limit (HTTP 429) or access error	Explicit error; no repeated retries		
T10	Missing `GOOGLE_API_KEY` or agent module	UI shows a diagnostic message instead of crashing		
T11	New chat button	Clears messages and starts a fresh ADK session		
T12	Save to Library	Item appears in Library with timestamp		

Metrics recommended for the observation table: average response time per query, number of tool calls per query, percentage of returned papers with abstracts, and percentage of DOIs that resolve when clicked.

6.3 Observations

- Because model and API calls are deliberately spaced (about 4 s and 3 s by default), a full multi-tool answer takes noticeably longer than a single chatbot reply. This trades speed for staying within free-tier quotas.
- Grounding in OpenAlex means every listed paper corresponds to a real record, and missing abstracts are reported honestly.
- The deterministic bibliography tool avoids malformed citations that an LLM might produce.

6.4 Limitations

1. **Metadata only.** OpenAlex provides metadata and abstracts, not full text; deep analysis of methods and results is limited to what the abstract states.
2. **Missing abstracts.** Some works have no abstract in OpenAlex.
3. **No independent verification.** Source verification confirms an OpenAlex record only; it does not prove peer review or publisher authenticity.
4. **APA 7 only.** Other citation styles are not implemented.
5. **Name parsing.** APA author formatting treats the last token as the surname, which can be wrong for compound or non-Western names; manual checking is advised.
6. **Keyword search.** Retrieval is keyword-based, not semantic, so relevant papers with different wording may be missed.
7. **In-memory state.** Sessions and the Library are stored in memory and are lost when the app restarts; there is no user login or database.
8. **Placeholder pages.** The Scheduled page is a reserved future feature.
9. **Model dependence.** Quality and speed depend on the Gemini free-tier model and its quotas; the LLM can still misjudge relevance.
10. **Single-process rate limiting.** The throttles apply within one Python process only.

7. Conclusion and Future Scope

7.1 Summary of Achievements

The project delivers a working agentic research assistant that plans a literature search, retrieves real scholarly records from OpenAlex, rebuilds abstracts, checks record existence, synthesises findings and produces APA 7 references. A custom Streamlit interface makes the agent usable by non-programmers, and design choices (tool-grounded answers, deterministic citation formatting, explicit error statuses, rate limiting) make it more trustworthy than an unguided chatbot.

7.2 Possible Improvements and Extensions

- Add more sources (Semantic Scholar, arXiv, Crossref, CORE) and merge duplicates.
- Add embedding-based semantic search and a vector store for true RAG over downloaded open-access PDFs.
- Support more citation styles (IEEE, MLA, Harvard) and BibTeX export.
- Persist the Library and chat history in a database with user authentication.
- Implement the Scheduled page for recurring alerts on new papers in a topic.
- Add citation-graph exploration and automatic research-gap detection.

- Add a multi-agent design (searcher, reviewer, writer) and automated evaluation of answer faithfulness.
- Export reports to PDF or Word.

8. References and GitHub Repository Link

1. Google. (n.d.). *Agent Development Kit (ADK) documentation*. https://google.github.io/adk-docs/
2. Google. (n.d.). *Gemini API documentation*. https://ai.google.dev/gemini-api/docs
3. OurResearch. (n.d.). *OpenAlex technical documentation*. https://docs.openalex.org
4. Priem, J., Piwowar, H., & Orr, R. (2022). OpenAlex: A fully-open index of scholarly works, authors, venues, institutions, and concepts. *arXiv*. https://arxiv.org/abs/2205.01833
5. Streamlit Inc. (n.d.). *Streamlit documentation*. https://docs.streamlit.io
6. Lewis, P., et al. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems, 33*. https://arxiv.org/abs/2005.11401
7. Yao, S., et al. (2023). ReAct: Synergizing reasoning and acting in language models. *ICLR 2023*. https://arxiv.org/abs/2210.03629
8. American Psychological Association. (2020). *Publication manual of the American Psychological Association* (7th ed.).

GitHub Repository: https://github.com/hyon99/Kognify.AI

Deployed Application: https://kognify-ai.streamlit.app